

Glan — Architecture & Pipelines de données

Document technique interne — version 1.0 — 17 mai 2026 Audience : équipe dev, architecte, intégrateurs techniques, partenaires.

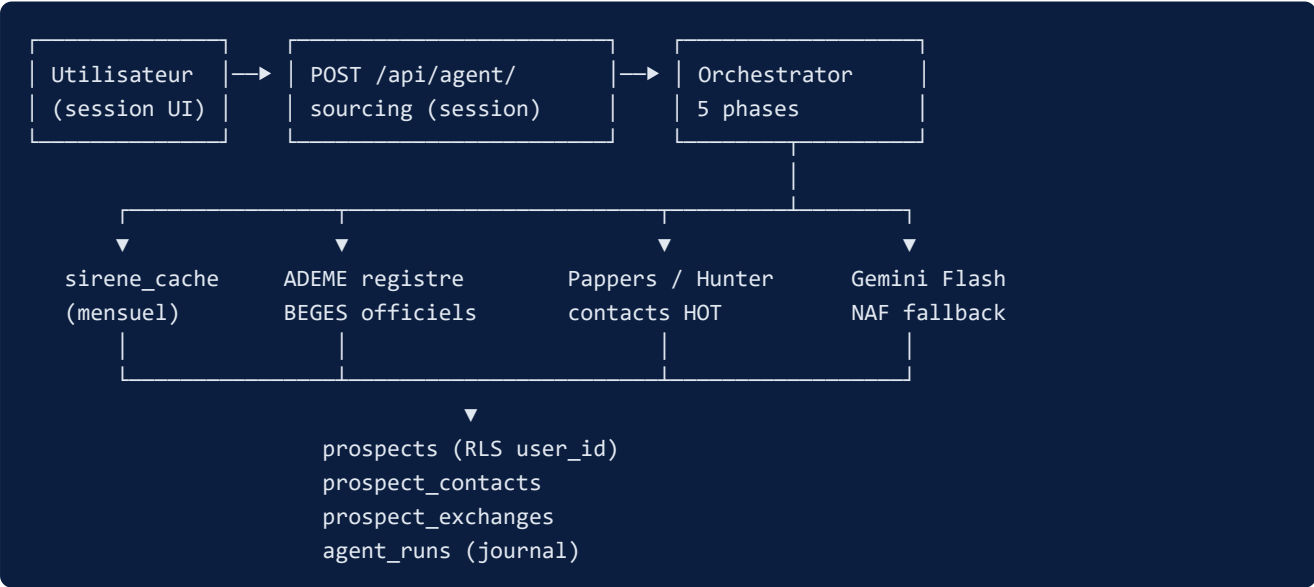
1. Vue d'ensemble

Glan est l'agent de prospection BEGES de l'application **ProspectionAgent**. Il automatise le sourcing, l'enrichissement et le scoring d'entreprises soumises ou éligibles au Bilan Gaz à Effet de Serre réglementaire (article L229-25 du code de l'environnement, France).

Stack

- **Next.js 15** (App Router, Turbopack) + **React 19** + **TypeScript 5 strict**
- **Supabase** (Postgres + Auth + Row Level Security)
- **Vercel** — hébergement applicatif + crons courts (≤ 300 s)
- **GitHub Actions** — job ETL bulk (hors cap Vercel)
- **APIs externes** : Sirene INSEE, ADEME, Recherche Entreprises (data.gouv), Pappers, Hunter.io, BODACC, INPI, Gemini Flash 2.0
- **OpenAI** (`gpt-4o`) — module legacy de pitch, désormais orphelin (à supprimer LOT 6)

Architecture logique



2. Pipeline « Run à la demande »

Déclenché par session utilisateur (`POST /api/agent/sourcing` ou `POST /api/agent/run`). Garde anti-concurrent : réponse `409` si un run `status='running'` existe déjà pour cet utilisateur. Chaque transition de phase est loggée dans `agent_runs.logs` (JSONB chronologique).

Phase 1 — Sourcing

- **Source primaire** : table partagée `sirene_cache` (alimentée par le pipeline mensuel, ~2 à 3 M établissements après filtre BEGES).
- **Fallback live** : API Sirene INSEE OAuth2 (~50 req/min, instabilité fréquente observée).
- **Fallback ultime** : API Recherche Entreprises data.gouv.fr (sans clé).
- **Pagination persistante** : curseur opaque stocké dans `profiles.sourcing_state` , reprise transparente au run suivant.
- **Volume par run** : `max(sourcing_target_per_run × 3, 50)` entreprises examinées.
- **Upsert idempotent** : batches de 50 sur clé `(user_id, siren)` .

Phase 2 — Enrichissement ADEME

- Endpoint `data.ademe.fr/.../publications-beges` .
- Batches **de 20 SIREN** pour respecter la pagination.
- 5 champs extraits par prospect :
 - `obligation_beges` — entreprise soumise à L229-25
 - `beges_publie` — bilan déposé au registre ADEME
 - `beges_valide` — encore dans sa fenêtre de 4 ans
 - `beges_date_publication` — date du dernier dépôt
 - `beges_url` — lien direct vers le rapport public
- Sentinelle : si l'ADEME répond 5xx sur > 30 % des batches, abort propre + remontée Sentry.

Phase 3 — Catégorisation secteur

Cascade en deux niveaux pour résoudre `secteur_libelle` à partir du code NAF :

1. **Table officielle NAF rév. 2 INSEE** (`lib/agent/naf-labels.ts`) — résolution instantanée gratuite.
2. **Gemini 2.0 Flash** en fallback uniquement si NAF introuvable. Parallélisme `SECTEUR_BATCH_PARALLELISM` . Plafond `SECTEUR_MAX_PER_RUN = 200` par run pour maîtriser le coût.

Phase 4 — Scoring

- Trois piliers : **taille**, **BEGES**, **contact**.
- Pondérations utilisateur via `profile.settings.scoring_weights` (migration 013). Défaut 33/34/33.
- Échelle finale `score_priorite` ∈ [0, 100].
- Tri `desc` → pipeline d'enrichissement contacts.

Phase 5 — Enrichissement contacts (stratégie HOT/COLD)

- **HOT** : `obligation_beges = true AND (beges_publie = false OR beges_valide = false)` → priorité dans la cascade payante : Pappers (téléphone + dirigeant), puis Hunter.io (email pattern).
 - **COLD** : autres → sources gratuites uniquement (Recherche Entreprises, INPI).
 - Quotas free tier : **Pappers 100 req/mois, Hunter 25 req/mois**.
 - Plafond `150 prospects/run`, appels **séquentiels** pour préserver les crédits.
 - Phase **non-fatale** : une erreur ici ne bloque pas le run.
 - Garde-fous RGPD :
 - Exclusion `statut = 'do_not_contact'` (opt-out manuel utilisateur).
 - Exclusion `statut = 'rejected'` (pas de double-tentative coûteuse).
 - Table `opt_out` partagée — jamais enrichi ni recontacté.
 - Un email perso (gmail, hotmail, yahoo...) **n'écasse jamais** un primary `is_pro = true`.
-

3. Pipeline mensuel — Cache SIRENE

Job exécuté **hors Vercel** (le cap `maxDuration` est insuffisant pour une ETL de 25-30 min), opéré par GitHub Actions.

Workflow : `.github/workflows/sirene-import.yml`

- Cron `0 4 1 * *` (1^{er} du mois 04:00 UTC, ≈ 05:00 Paris hiver / 06:00 été).
- Environment GitHub `Production - strofe-prospection-agent` (secrets injectés automatiquement).
- Timeout 60 minutes (durée observée ~25-30 min).

Script : `scripts/import-sirene-bulk.ts`

1. Résolution **dynamique** de la ressource via `https://www.data.gouv.fr/api/1/datasets/base-sirene.../` — les `resource_id` changent à chaque republication mensuelle.
 2. GET streamé du ZIP (~1,1 GB compressé, ~3 GB décompressé) vers `/tmp`, retry exponentiel ×3 sur `5xx/timeout`.
 3. `unzipper` en flux + `csv-parse` ligne par ligne.
 4. Filtre BEGES strict (early-skip) appliqué à la volée :
 - `etatAdministratifEtablissement = 'A'` (actif)
 - `etablissementSiege = 'true'` (siège uniquement)
 - `trancheEffectifsEtablissement ∈ {21..53}` (≥ 10 salariés)
 - Section NAF ∈ {A, C, D, E, F, H} (Agri / Industrie / Énergie / Eau-déchets / Construction / Transport)
 5. Upsert `sirene_cache` par batches de 500, `onConflict siren` — idempotent, rejouable.
 6. Garde-fou storage : toutes les 10 000 lignes, query `sirene_cache_size`, abort si > `MAX_STORAGE_MB` (défaut 100 MB pour rester sous Free Tier Supabase 500 MB).
 7. Skip intelligent : `/api/admin/sirene-status` retourne `days_since_import` ; si < 25 j, l'import est sauté. Forçage manuel via `workflow_dispatch` avec input `force=true`.
-

4. Pipelines automatisés annexes

4.1 reap-stale — quotidien 04:00 UTC

Cron Vercel `0 4 * * *` sur `/api/agent/reap-stale`. Détecte les runs `status='running'` qui dépassent **60 minutes** (zombies post-crash ou timeout réseau) et les marque `status='failed'` avec `error_message = 'reaped: exceeded timeout'`.

Idempotent. Garantit qu'un utilisateur bloqué peut relancer un run au cycle suivant sans intervention humaine.

4.2 bodacc-changes — hebdo lundi 04:00 UTC (à activer dans `vercel.json`)

Cron recommandé `0 4 * * 1`. Source : `bodacc-datadila.opendatasoft.com` (registre légal des annonces commerciales).

- Scan glissant **8 jours** (chevauchement intentionnel pour ne rien manquer).
- Pour chaque SIREN touché par un changement de dirigeant, met à jour `prospects.contact_outdated_at = NOW()`. Le contact reste lisible, mais signalé périmé dans l'UI.
- Batches de 50 SIREN par requête.
- Service role (bypass RLS, scan multi-utilisateurs).
- **Aucun nom de dirigeant loggé** (minimisation RGPD malgré le caractère public BODACC).

4.3 purge-old-prospects — quotidien 03:00 UTC (à activer dans `vercel.json`)

Cron recommandé `0 3 * * *`. Supprime les prospects au-delà de 3 ans :

- `last_enrichment_run_at < NOW() - 3 ans`, OU
- `last_enrichment_run_at IS NULL AND created_at < NOW() - 3 ans`.

CASCADE : `prospect_contacts`, `prospect_exchanges`, `prospect_notes` supprimés via FK `ON DELETE CASCADE`.

Conformité RGPD CNIL : durée de conservation proportionnée à la finalité commerciale.

4.4 Synthèse calendrier des automatismes

Pipeline	Fréquence	Plateforme	Statut
<code>reap-stale</code>	Quotidien 04:00 UTC	Vercel	Actif
<code>agent/run legacy multi-users</code>	Quotidien 22:00 UTC	Vercel	Actif (à arbitrer post-pivot)
<code>bodacc-changes</code>	Hebdo lundi 04:00 UTC	Vercel	À activer
<code>purge-old-prospects</code>	Quotidien 03:00 UTC	Vercel	À activer
<code>sirene-import</code> (bulk INSEE)	Mensuel 1 ^{er} 04:00 UTC	GitHub Actions	Actif

5. Modèle de données

Table	Rôle	Volume cible
profiles	Settings JSONB (offre, NAF cibles, géo, scoring_weights, sourcing_state)	1 ligne / user
prospects	Entreprises sourcées + BEGES + score + statut Kanban	~1 000 / user
prospect_contacts (mig. 011)	Multi-contacts par prospect (DAF, RSE, DG)	3 à 5 / prospect
prospect_exchanges (mig. 012)	Journal échanges horodatés (appel, email, LinkedIn, RDV)	5 à 10 / prospect
prospect_notes	Notes libres utilisateur	ad-hoc
agent_runs	Journal cron (phase, compteurs, logs JSONB)	1 / run
sirene_cache	Table publique partagée, rafraîchie mensuel	~2 à 3 M lignes
opt_out	SIREN/emails refusant la prospection (RGPD)	partagé

Pipeline commercial : 8 statuts — sourced → qualified → contacted → interested → offer_sent → converted + rejected + on_hold . Statut rdv legacy conservé en base, retiré de l'UI.

6. Sécurité

- **RLS Postgres** sur les 5 tables utilisateur — 4 policies (SELECT/INSERT/UPDATE/DELETE) par table, prédicat strict `auth.uid() = user_id` . Pas de double-filtre applicatif en code authentifié.
- **Service role** (SUPABASE_SERVICE_ROLE_KEY) isolée aux routes pipeline (`app/api/agent/*` , `app/api/cron/*` , scripts ETL). **Aucun import** toléré depuis `app/(dashboard)/` ou `components/` .
- **CRON_SECRET** : comparaison `crypto.timingSafeEqual` avec padding 128 octets dans `lib/auth/cron.ts#isCronRequest` . Jamais loggé, jamais comparé via `===` .
- **Whitelist SSRF** : `fetch()` externes restreints à 9 domaines (`api.insee.fr` , `data.ademe.fr` , `recherche-entreprises.api.gouv.fr` , `api.pappers.fr` , `api.hunter.io` , `api.resend.com` , `bodacc-datadila.opendatasoft.com` , `registre-national-entreprises.inpi.fr` , `www.data.gouv.fr`).
- **Validation Zod** systématique aux frontières (bodies API entrants, réponses des APIs externes).
- **Sentry beforeSend** : scrub des emails et téléphones de prospects avant remontée d'erreur.

7. Observabilité

- Chaque run écrit dans `agent_runs` : phase , compteurs (`prospects_sourced` , `prospects_qualified` , `prospects_enriched`), logs JSONB chronologique, `error_message` si échec.

- Sentry capture les erreurs serveur + perf web vitals.
 - Page **Pipeline** (UI) lit `agent_runs` et affiche un Sankey des phases + journal détaillé par étape.
-

8. Annexes

Commandes utiles

```
npm run dev           # Next dev (Turbopack)
npm run build          # Production build
npm run type-check     # tsc --noEmit
npm run test           # Vitest run
npm run import-sirene  # ETL bulk SIRENE local (Node 22.6+)
npx supabase migration new <nom>
npx supabase db push

# Test cron local
curl -H "Authorization: Bearer $env:CRON_SECRET" \
  -X POST http://localhost:3000/api/agent/run
```

Points d'entrée code

- Orchestrator : `lib/agent/orchestrator.ts`
 - Sourcing runner : `lib/agent/sourcing-runner.ts`
 - ETL bulk SIRENE : `scripts/import-sirene-bulk.ts`
 - Workflow mensuel : `.github/workflows/sirene-import.yml`
 - Crons Vercel : `vercel.json`
 - Cron auth helper : `lib/auth/cron.ts`
-

— Équipe Glan
